

Perguntas e Respostas de Entrevistas de Dev Senior

.NET

Qual a diferença entre .NET Framework, .NET Core, e .NET?

O que é Intermediate Language (IL)?

Como a compilação Just-in-Time (JIT) funciona?

O que é Common Language Runtime (CLR)?

Como funciona o gerenciamento de memória no .NET?

O que é o Garbage Collector (GC)?

Qual a diferença entre os tipos de valor e tipos de referência?

O que são REST APIs?

Quais os tipos de caching e como funcionam?

Quais ações de melhoria em performance podemos fazer em uma API?

O que é injeção de dependência e quais os tempos de ciclo de vida disponíveis?

Quais as diferenças entre EF Core e Dapper?

ASP.NET

Clean Code e Arquitetura de Software

O que é mensageria?

O que é CQRS?

O que é Event Sourcing?

Como escolher entre bancos de dados SQL e NoSQL?

O que são os princípios SOLID? Dê exemplos de cada um.

O que são Design Patterns? Cite alguns que você utiliza/utilizou em seus projetos.

O que é POO? Quais os seus pilares?

O que é DRY e YAGNI?

Qual diferença entre composição e herança?

O que é um API Gateway?

Como funciona caching em uma aplicação distribuída?

Quais as principais diferenças entre uma arquitetura monolítica e de microsserviços?

Linguagem C#

O que é uma expressão lambda?

Qual a diferença entre uma classe abstrata e uma interface?

O que é e como criar um método de extensão?

Qual o propósito da instrução using?

Qual a diferença entre overload e override?

Qual a diferença entre IEnumerable e IQueryable?

O que acontece quando se chama ToList em uma coleção?

O que é LINQ?

O que é e como funciona generics?

O que é async e await?

O que são Threads e Tasks?

Como é possível aguardar a execução de múltiplas Tasks?

Perguntas e Respostas de Entrevistas de Dev Senior

1. Clean Code e Arquitetura de Software

1.1. O que é mensageria?

Mensageria é uma técnica de comunicação entre sistemas que permite o envio e recebimento de mensagens de forma assíncrona. Em uma arquitetura orientada a eventos, por exemplo, mensageria é usada para desacoplar a comunicação da aplicação com serviços externos e outros componentes do sistema.

Diferença conceitual

- Mensagem => vai ser consumida uma única vez
- Evento => poderá ser consumido por um ou mais inscritos

1.2. O que é CQRS?

CQRS é padrão arquitetural que separa as operações de leitura e escrita de um sistema, utilizando diferentes modelos de domínio para cada uma delas. O objetivo é melhorar a escalabilidade e o desempenho da aplicação, permitindo que as operações de leitura sejam otimizadas para consultas e as operações de escrita para atualizações.

1.3. O que é Event Sourcing?

Event Sourcing é uma técnica de arquitetura de software que consiste em armazenar todos os eventos que ocorrem em uma entidade, em vez de armazenar apenas o estado atual. Isso permite a reconstrução do estado atual a partir dos eventos, facilitando a implementação de recursos como desfazer e refazer e histórico de alterações.

1.4. Como escolher entre bancos de dados SQL e NoSQL?

A escolha entre bancos de dados SQL e NoSQL depende das necessidades do projeto e das características dos dados que serão armazenados. Bancos de dados SQL são indicados para projetos que requerem integridade e consistência dos dados, e onde é importante garantir a conformidade com as restrições e regras de negócio. Bancos de dados NoSQL, por outro lado, são indicados para projetos que requerem escalabilidade horizontal, onde é necessário lidar com grandes quantidades de dados e onde a consistência dos dados pode ser eventual em favor da disponibilidade e desempenho.

1.5. O que são os princípios SOLID? Dê exemplos de cada um.

Os princípios SOLID são um conjunto de cinco princípios de desenho de software que ajudam a criar um código mais limpo, fácil de manter e extensível.

Eles incluem o Princípio da Responsabilidade Única (SRP), que estabelece que uma classe deve ter apenas uma razão de ser alterada; o Princípio Aberto-Fechado (OCP), que estabelece que uma classe deve estar aberta para extensão, mas fechada para modificação; o Princípio da Substituição de Liskov (LSP), que estabelece que uma classe derivada deve ser substituível por sua classe base; o Princípio da Segregação de Interfaces (ISP), que estabelece que uma interface deve ter apenas métodos que são relevantes para seus clientes; e o Princípio da Inversão de Dependência (DIP), que estabelece que os módulos de alto nível não devem depender dos módulos de baixo nível, mas sim de abstrações. Cada um desses princípios ajuda a criar um código mais flexível, extensível e de fácil manutenção.

1.6. O que são Design Patterns? Cite alguns que você utiliza/utilizou em seus projetos.

Design Patterns são soluções recorrentes para problemas de desenho de software, que foram desenvolvidas e documentadas ao longo dos anos por programadores experientes. Eles são soluções comprovadas que podem ser aplicadas em diferentes contextos para resolver problemas comuns. Exemplos de Design Patterns incluem Singleton, Factory Method, Strategy, Façade, Builder, entre outros.

1.7. O que é POO? Quais os seus pilares?

POO (Programação Orientada a Objetos) é um paradigma de programação que se concentra na abstração de objetos e suas interações. Os pilares da POO são: encapsulamento, herança, polimorfismo e abstração. O encapsulamento é o princípio de ocultar a complexidade interna de um objeto, expondo apenas uma interface pública. A herança permite a criação de novas classes a partir de classes existentes, compartilhando comportamentos e propriedades. O polimorfismo permite que objetos de diferentes classes sejam tratados como se fossem do mesmo tipo, permitindo a execução de implementações de mesmo método de cada uma dessas classes. A abstração é o processo de abstrair um conceito, comportamento ou entidade em uma classe ou objeto no código.

1.8. O que é DRY e YAGNI?

DRY (Don't Repeat Yourself) é um princípio de programação que enfatiza a importância de evitar a repetição de código. YAGNI (You Ain't Gonna Need It) é outro princípio que enfatiza a importância de não adicionar funcionalidades ou recursos que não são estritamente necessários no momento. Ambos os princípios visam a simplificar e reduzir a complexidade do código, promovendo a manutenibilidade e a flexibilidade.

1.9. Qual diferença entre composição e herança?

A composição é um princípio de programação que consiste em criar objetos complexos a partir de objetos mais simples, através de associações entre eles. A herança, por outro lado, é um princípio que permite que as classes sejam criadas a partir de outras classes existentes, compartilhando seus comportamentos e propriedades. A principal diferença entre os dois é que a herança é uma relação "é um", enquanto a composição é uma relação "tem um". A herança pode tornar o código mais fácil de entender e manter, mas também pode levar a problemas de acoplamento e complexidade excessiva.

1.10. O que é um API Gateway?

Um API Gateway é um componente de software que funciona como ponto de entrada para um conjunto de microserviços ou APIs. Ele fornece uma interface unificada para os clientes, permitindo que eles acessem múltiplas APIs ou serviços através de um único ponto de entrada. O API Gateway também pode ser usado para gerenciar autenticação, autorização, balanceamento de carga, roteamento, cache e monitoramento.

1.11. Como funciona caching em uma aplicação distribuída?

O caching em uma aplicação distribuída funciona armazenando dados frequentemente acessados em memória ou em um cache externo, como o Redis. Dessa forma, quando um usuário solicita um recurso, o cache é verificado primeiro para ver se ele já foi armazenado. Se o recurso estiver presente no cache, ele é retornado imediatamente. Se não, esse recurso precisará ser buscado em sua origem, e guardado na cache para as próximas requisições.

Isso reduz o número de solicitações que são enviadas para o servidor e melhora a performance da aplicação.

1.12. Quais as principais diferenças entre uma arquitetura monolítica e de microsserviços?

A arquitetura monolítica é uma abordagem de desenvolvimento de software em que todas as funcionalidades de uma aplicação são agrupadas em um único monólito. Essa abordagem torna mais fácil o desenvolvimento de pequenas aplicações que não possuem muitas complexidades e não precisam de escalabilidade, mas dificulta o processo de manutenção e atualização de aplicações grandes. Além disso, alterações em uma parte da aplicação podem ter impacto em outras partes.

Já a arquitetura de microsserviços consiste na divisão de uma aplicação em componentes menores e independentes, cada um executando um conjunto de funcionalidades específicos e relacionados (comumente através de Agregados). Esses componentes são desenvolvidos e implantados separadamente, o que torna a manutenção e atualização mais fácil. Além disso, o uso de microsserviços permite a escalabilidade individual de cada componente, melhorando a performance e a disponibilidade da aplicação. Entretanto, essa abordagem traz desafios adicionais em termos de comunicação, orquestração e segurança.

2. Linguagem C#

2.1. O que é uma expressão lambda?

Expressão lambda é um recurso da linguagem que permite criar funções anônimas de maneira concisa. As expressões lambda são usadas para passar código como parâmetros de outras funções ou métodos. Elas são frequentemente usadas para definir predicados, comparadores e transformações de coleções em uma sintaxe mais concisa e legível.

2.2. Qual a diferença entre uma classe abstrata e uma interface?

Tanto a classe abstrata quanto a interface são mecanismos de abstração em C#. A diferença principal entre elas é que uma classe abstrata pode conter implementações de métodos, enquanto uma interface só pode definir as assinaturas de métodos. Além disso, uma classe pode implementar várias interfaces, mas só pode herdar de uma classe abstrata. Já uma classe abstrata pode ter construtores, campos e propriedades, enquanto uma interface só pode ter propriedades, métodos e eventos.

2.3. O que é e como criar um método de extensão?

Métodos de extensão permitem adicionar novos métodos a tipos existentes sem precisar criar uma nova classe derivada. Eles são criados criando-se uma classe estática e um método estático, que aceita o tipo que se deseja estender como primeiro parâmetro e utiliza a palavra-chave "this" na definição do parâmetro para indicar que se trata de um método de extensão. Métodos de extensão são úteis para adicionar comportamentos específicos a tipos de bibliotecas externas ou para estender tipos base da linguagem.

2.4. Qual o propósito da instrução using?

A instrução using é usada para gerenciar recursos não gerenciados em C#. Ela permite que um objeto que implemente a interface IDisposable seja usado dentro de um bloco de código, e ao final do bloco, o método Dispose do objeto é chamado automaticamente. Isso garante que os recursos utilizados pelo objeto sejam liberados, mesmo se ocorrerem exceções durante a execução do bloco de código.

2.5. Qual a diferença entre overload e override?

Overload e override são conceitos relacionados a polimorfismo. Overload se refere à criação de vários métodos com o mesmo nome, mas com diferentes parâmetros, o que permite que diferentes sobrecargas do método sejam chamadas dependendo dos argumentos passados. Override se refere à criação de um método em uma classe derivada que substitui a implementação de um método com o mesmo nome na classe base. O método substituto deve ter a mesma assinatura que o método original e deve usar a palavra-chave "override".

2.6. Qual a diferença entre IEnumerable e IQueryable?

IEnumerable e IQueryable são interfaces usadas para consultar coleções de dados em C#. IEnumerable é usada para coleções em memória, como listas, e fornece métodos para enumerar a coleção. IQueryable é usada para consultar coleções de dados que podem estar armazenadas fora da memória, como em um banco de dados. IQueryable permite a construção de consultas de forma mais eficiente, enviando as consultas para o servidor de banco de dados de forma mais otimizada.

2.7. O que acontece quando se chama ToList em uma coleção?

Quando você chama o método `ToList()` em um objeto do tipo `IEnumerable`, todos os dados na coleção são carregados na memória do sistema e são manipulados pela aplicação. O resultado é que todas as operações de consulta são executadas em memória. Isso pode ser problemático para grandes conjuntos de dados, pois pode levar a problemas de desempenho e limitações de memória.

Já ao chamar o método `ToList()` em um objeto do tipo `IQueryable`, o LINQ gera uma consulta SQL a partir das expressões de consulta definidas no código e envia essa consulta para o banco de dados. O banco de dados, então, retorna apenas o subconjunto de dados solicitado pela consulta. A consulta só é executada quando você chama o método `ToList()`, que traz os dados do banco de dados para a memória. Esse processo deixa a execução muito mais rápida, especialmente em conjuntos de dados grandes.

2.8. O que é LINQ?

LINQ é um componente da Microsoft que permite a manipulação de dados de forma consistente, com duas abordagens: query-based e method-based.

O query-based utiliza uma sintaxe de consulta semelhante ao SQL, enquanto o method-based utiliza chamadas de métodos que podem ser encadeadas.

Com o LINQ, é possível realizar consultas em coleções de objetos, bancos de dados e serviços web, permitindo que as operações sejam executadas de forma mais rápida e simples.

2.9. O que é e como funciona generics?

Generics permitem que tipos de dados sejam definidos em um momento posterior à criação do código, oferecendo maior flexibilidade e reutilização de código. A principal vantagem é que as classes e métodos genéricos podem ser reutilizados com tipos diferentes, sem a necessidade de reescrever o código para cada tipo de dado.

2.10. O que é async e await?

Async e Await são recursos do C# que permitem que as operações assíncronas sejam executadas de forma mais eficiente, sem a necessidade de bloquear a thread que está executando aquele código. Quando uma operação é marcada como "await", a execução do código é pausada até que a operação assíncrona seja concluída, permitindo que outras operações sejam executadas em paralelo. Isso resulta em um aplicativo mais responsivo e com melhor desempenho.

2.11. O que são Threads e Tasks?

Threads são fluxos de execução de código que permitem que múltiplas tarefas sejam executadas simultaneamente. As tasks são um tipo de thread que foram introduzidas no .NET Framework 4.0 e são uma maneira mais fácil e eficiente de executar operações em segundo plano. As tasks usam menos recursos do sistema e são mais flexíveis do que as threads, permitindo que o código seja executado em paralelo e com melhor desempenho.

2.12. Como é possível aguardar a execução de múltiplas Tasks?

É possível aguardar a execução de múltiplas tasks usando o método `Task.WhenAll`. Esse método permite que várias tasks sejam executadas em paralelo e aguarda até que todas sejam concluídas. Um exemplo de código seria:

```
var task1 = ChamadaDeAPIUm();  
var task2 = ChamadaDeAPIDois();  
var task3 = ChamadaDeAPITres();  
await Task.WhenAll(task1, task2, task3);
```

3. ASP.NET

3.1. O que são REST APIs?

REST (Representational State Transfer) é um estilo arquitetural de desenvolvimento de APIs, que utiliza o protocolo HTTP para expor operações que podem ser executadas em recursos específicos. O objetivo é fornecer uma interface uniforme para acesso aos recursos, independente de plataforma ou linguagem de programação utilizada.

3.2. Quais os tipos de caching e como funcionam?

Existem dois tipos principais de caching: server-side e client-side. O server-side caching é feito pelo servidor que hospeda a API, e consiste em armazenar o resultado de uma requisição para fornecer a mesma resposta quando a mesma requisição é feita novamente. O client-side caching é realizado pelo cliente, e consiste em armazenar a resposta recebida para a mesma requisição, para que não precise ser feita novamente.

Existem dois tipos de caching server-side: cache de memória e cache distribuído. O cache de memória armazena os dados em memória RAM e é acessado rapidamente, permitindo uma melhoria significativa de performance em consultas repetidas. Já o cache distribuído armazena os dados em uma infraestrutura distribuída, como um banco de dados em memória ou um servidor de cache dedicado, permitindo compartilhar os dados entre várias instâncias de um aplicativo e melhorando a escalabilidade horizontal. O ASP.NET Core oferece suporte nativo a ambos os tipos de caching e, dependendo do cenário, pode ser necessário escolher uma ou outra opção.

3.3. Quais ações de melhoria em performance podemos fazer em uma API?

Para melhorar o desempenho de uma API ASP.NET Core, algumas ações incluem: otimização de banco de dados, redução de latência de rede, diminuir quantidade de requisições externas, melhoria na performance de acesso a serviços externos, utilização de caching server-side (como memory cache e cache distribuído), e otimização de código com multi-threading e/ou paralelismo, e algoritmos.

Porém, vale lembrar que a otimização da performance não é uma ação única, mas sim um processo contínuo, que deve ser avaliado e aprimorado constantemente. É importante analisar e entender os gargalos de performance da API e aplicar as ações que melhor atendam às necessidades do projeto.

3.4. O que é injeção de dependência e quais os tempos de ciclo de vida disponíveis?

Injeção de dependência é um padrão de projeto que permite o gerenciamento de dependências de uma aplicação. No ASP.NET Core, ele é utilizado para injetar instâncias de classes em outras classes, sem que seja necessário criar as instâncias manualmente. Existem três tipos de ciclo de vida de injeção de dependência: Transient (a cada instância), Scoped (por solicitação) e Singleton (uma única instância).

3.5. Quais as diferenças entre EF Core e Dapper?

EF Core e Dapper são duas bibliotecas para acesso a banco de dados em .NET. EF Core é um ORM (Object Relational Mapper) completo, que abstrai o acesso ao banco de dados por meio de classes. Dapper é uma biblioteca mais simples, que utiliza consultas SQL para realizar operações no banco de dados. EF Core é mais abrangente e pode ser mais fácil de usar em projetos maiores, enquanto o Dapper é mais rápido e pode ser mais adequado para projetos menores, mas também tem seus usos em projetos maiores principalmente por ser performático.

4. .NET

4.1. Qual a diferença entre .NET Framework, .NET Core, e .NET?

O .NET Framework é uma implementação do .NET original que roda apenas em sistemas operacionais Windows, enquanto o .NET Core é uma versão multiplataforma do .NET que roda em várias plataformas, incluindo Windows, macOS e Linux. .NET é uma família de tecnologias e ferramentas que inclui o .NET Framework e o .NET Core, bem como outras implementações, como o .NET 5 e o .NET Standard.

O .NET Standard é uma especificação que define as APIs que devem estar presentes em todas as implementações do .NET, independentemente da plataforma de destino. O objetivo do .NET Standard é permitir que os desenvolvedores criem bibliotecas .NET que possam ser consumidas por diferentes implementações do .NET, incluindo o .NET Framework, o .NET Core e o Xamarin.

4.2. O que é Intermediate Language (IL)?

Intermediate Language (IL) é uma linguagem de baixo nível que é gerada pelo compilador do .NET a partir do código-fonte. O IL é executado pelo CLR e pode ser considerado uma forma de código executável intermediário entre o código-fonte e o código de máquina.

4.3. Como a compilação Just-in-Time (JIT) funciona?

A compilação Just-in-Time (JIT) é um processo de compilação que o CLR usa para converter o código IL em código de máquina nativo durante o tempo de execução. O CLR compila cada método em um assembly .NET em tempo de execução, à medida que o método é executado pela primeira vez. A compilação JIT pode melhorar o desempenho do tempo de execução, pois permite que o CLR otimize o código para a plataforma específica em que está sendo executado.

4.4. O que é Common Language Runtime (CLR)?

O Common Language Runtime (CLR) é a máquina virtual responsável por executar o código gerenciado, independentemente da implementação específica do .NET. Ele fornece recursos como gerenciamento de memória, segurança, coleta de lixo e gerenciamento de exceções.

O Common Language Runtime (CLR) é o componente do .NET Framework (e outras implementações do .NET) que executa o Intermediate Language (IL). Quando um programa é compilado em C# ou outra linguagem .NET, o compilador gera um arquivo executável (como um .exe ou .dll) que contém código IL em vez de código de máquina nativo. Durante a execução do programa, o CLR carrega o arquivo executável e o traduz em tempo de execução para o código de máquina nativo do sistema em que o programa está sendo executado. Isso é conhecido como compilação Just-in-Time (JIT) e é uma das maneiras pelas quais o CLR ajuda a garantir a segurança e a interoperabilidade do código .NET. Então, em resumo, o CLR executa o Intermediate Language (IL), que é traduzido em tempo de execução para código de máquina nativo.

4.5. Como funciona o gerenciamento de memória no .NET?

O gerenciamento de memória no .NET é realizado por meio de um coletor de lixo (garbage collector). O garbage collector é responsável por identificar e coletar automaticamente os objetos não utilizados na memória. Isso permite que os desenvolvedores não precisem gerenciar manualmente a alocação e a liberação de memória. O garbage collector opera de maneira assíncrona, periodicamente, e tem como objetivo manter a alocação de memória em níveis eficientes.

4.6. O que é o Garbage Collector (GC)?

O Garbage Collector (GC) é um recurso da Common Language Runtime (CLR) responsável por gerenciar a memória utilizada por um programa .NET. Ele é responsável por alocar e desalocar memória conforme necessário, evitando que o programa sofra de problemas como memory leaks e memory fragmentation. O GC funciona em três camadas: a camada de geração 0, a camada de geração 1 e a camada de geração 2.

Na camada de geração 0, os objetos recém-criados são alocados. A maioria dos objetos é desalocada rapidamente nessa camada, quando eles saem do escopo, o que a torna a camada com menor tempo de vida para objetos na memória. Já a camada de geração 1 é a área intermediária para objetos que sobrevivem à coleta da geração 0. E, por fim, a camada de geração 2 é a área para objetos de longa duração que sobrevivem às duas primeiras coletas. O GC opera de forma inteligente e periódica em cada camada para garantir que a memória seja gerenciada da melhor forma possível.

4.7. Qual a diferença entre os tipos de valor e tipos de referência?

Os tipos de valor armazenam diretamente o valor na memória, enquanto os tipos de referência armazenam um ponteiro para a memória em que o valor está localizado. Os tipos de valor são armazenados na pilha (stack) e são gerenciados pelo sistema operacional, enquanto os tipos de referência são armazenados no heap, que é uma região de memória gerenciada pelo Garbage Collector.

A alocação de tipos de valor é mais rápida, mas o uso de tipos de referência é mais flexível e eficiente para estruturas de dados complexas.